Beyond Reproducibility: On the Feasibility and Benefits of Extensive Evaluation During Artifact Evaluation

Abstract

Benchmark and system parameters often have a significant impact on performance evaluation, which raises a long-lasting question about which settings we should use.

This paper studies the feasibility and benefits of extensive evaluation, by reproducing 10 prior works and testing them under a wider range of settings. We make two contributions: first, we show that sampling and regression is effective to predict performance for well-engineered systems. Based on this observation, to reduce the cost of extensive evaluation, we suggest gradually testing more settings until the prediction accuracy becomes stable. Second, we design and develop a tool to visualize high-dimensional data from extensive evaluation on a figure called Tiled Heat Map.

Our experience shows that extensive evaluation can achieve two goals: first, it has revealed many correctness and performance issues, and addressing them can improve the quality of artifact and facilitate future comparison. Second, extensive evaluation can generate a more comprehensive comparison among different works.

1 Introduction

Through the emphasis on artifact evaluation, our community has greatly improved the quality of artifacts and simplified the process of reproducing and comparing to prior results.

However, we observe one long-lasting question persists despite the improvement of reproducibility: which settings (i.e., combinations of parameter values) should we use during our experiments? Popular benchmarks like TPC-C [69] and YCSB [20,78] and the systems to evaluate often have multiple parameters; as shown in prior works [35, 43, 45, 77], it often happens that one system can outperform another under one setting but not so under a different setting. So far our artifact evaluation only requires reproducing results under the same settings as reported in the corresponding publication, but does not require or recommend any settings to use.

In general, there exist two philosophies to answer this question. First, we can focus on one or a few representative settings, usually extracted from studies of production systems. Second, we can perform an extensive evaluation to cover all/many combinations of parameter values. Extensive evaluation has the obvious benefit that it can provide a comprehensive picture of system performance, which can avoid potential bias introduced by parameter selection, but it may cost a large amount of time and resources.

Which approach is more appropriate perhaps requires a thorough discussion in our community, but this paper tries to advocate extensive evaluation due to the following two observations: first, a number of studies have shown that industrial companies can have a variety of workloads [11, 14, 55, 79]. For example, Facebook finds "object sizes spanning more than seven orders of magnitude" for certain applications [14]. Such observation naturally calls for an extensive evaluation to reveal the full picture of system performance under different settings. Second, we observe a later research work often needs to compare with a prior work under a new setting that was not tested by the prior work. Our experience shows that this often incurs many problems, usually due to bugs or performance issues that only manifest under the new settings (Section 5); a number of works choose to re-implement prior works (e.g., Janus [53] re-implements TAPIR [82]; Aria [45] re-implements Calvin [68], etc)—this is hurting the purpose of artifact evaluation, i.e., "easy to reuse, facilitating further research" [4] to some degree. An extensive evaluation can help a researcher to identify and fix such problems.

This paper studies the feasibility and benefits of extensive evaluation. It tries to answer the following questions: 1) Can extensive evaluation help us to improve the quality of artifacts and allow a more comprehensive performance comparison? 2) In terms of time and resources, is it feasible to carry out extensive evaluation? And if not, how can we address that?

To answer these questions, we focus on TPC-C and YCSB, two of the most popular benchmarks in our community, because there is a general consensus about what parameters can be tuned for these two benchmarks. We selected 10 research prototypes evaluated under these two benchmarks. In addition to reproducing their results as reported in their original papers,

we further run settings that are not used in these papers: when we see failed tests or significant performance degradation, we try our best effort to diagnose and fix the problems. From such experience, we try to answer the above questions in terms of both system correctness and performance.

Correctness. In our experiments, it is not uncommon for a research prototypes to stop working under untested settings. However, since even mature commercial softwares are not free of bugs, it is perhaps unrealistic to expect a research prototype to be stable and reliable under all conditions, so the question we want to investigate here is whether it is possible to improve the quality of artifact with a reasonable amount of effort.

We tried to diagnose and fix the problems we met and observe the problems can be categoried into two types: the first type deterministically blocks an experiment under a certain setting. They are usually caused by hard-coded configurations. For example, a prototype may preallocate a fixed amount of memory, which will not work if we test the prototype with more data. Naturally, this type of problems are more likely to happen when we push the system to the extreme (e.g., a lot of data or a lot of nodes). Diagnosing this type of problems could take a while for us, since the error messages do not always directly point to the root cause, but should be easier for the authors who are familiar with the system; fixing this type of problems usually only requires changing a few lines of code. The second type can non-deterministically cause the prototype to crash during certain experiments. While we try our best to diagnose this type, we only succeed in a subset of them. This type of problems can happen under any settings, which means identifying them will need a large amount of effort, not to mention diagnosing and fixing them.

From such experience, we conclude that, while expecting a research prototype to be almost bug-free is unrealistic, fixing the first type of problems is both beneficial and feasible: it allows a future research work to compare with a prior work under a different setting, without requiring the additional effort of fixing the code of prior work; these types of problems can usually be triggered by running extreme settings and can usually be fixed easily, which means the authors do not need to spend a significant amount of effort to test different settings and diagnose the problems.

Performance. When comparing system performance under a wide range of settings, we observe extensive evaluation can bring two benefits:

First, extensive evaluation has revealed design issues that cause performance degradation in previously untested settings. For example, DrTM [76] and Silo experience performance degradation when loaded with more data, since their tree indexes grow larger. This problem could be addressed by partitioning data [16, 21, 40, 42]. Revealing and addressing these issues can avoid the problem that a later work has to "optimize" or "re-implement" a prior work for a fair comparison under a new setting. Furthermore, we also observe performance degradation that we do not know how to address, which could be future research topics as well.

Second, extensive evaluation allows us to perform a more comprehensive comparison of system performance under different settings. Perhaps not surprisingly, we find prior works sometimes report settings they excel at: while these settings may be important, it may not be a good idea to completely ignore other settings, as discussed above.

The cost, however, is significant: it tooks us thousands of machine hours to test one prototype, despite that we carefully chose the parameters to tune and the parameter values to use. For systems with more parameters, the cost could get out of control.

To address this challenge, we investigate the following question: is it possible to identify all such performance patterns by running a subset of settings?

.....

2 Methodologies

We focus on TPC-C and YCSB, two of the most popular benchmarks used in our community, because there is a general consensus about what parameters to tune for these two. Then among the works that are evaluated under these two benchmarks, we selected 10 that we can successfully reproduce, as listed in Table 1 and Table 2 (note that some systems are evaluated under both). Since our hardware is not exactly the same as those used by these works, we consider "success" as our reproduced numbers are reasonably close to those in the original paper and the conclusion of the paper still holds. For these 10 works, we are able to reproduce most of their results reported in the corresponding papers. In this work, we mainly focus on the throughput of these systems and we plan to explore latency in the future.

Note that this list is by no means a complete list of all reproducible works: we have to skip ones that require special hardware; we have not got a chance to try most recent publications, which we plan to explore in the future.

For a very brief introduction, among these 10 works, Silo and Cicada are single-node in-memory transaction processing engines; Calvin, DrTM, GAM, Star, and Aria support distributed transactions; TAPIR and Janus further support geo-distributed transactions; HERD is a distributed key-value store. DrTM, GAM, and HERD use RDMA techniques to improve performance.

After reproducing the original results of these systems, we try to test them under a broader range of settings.

Parameters of TPC-C. TPC-C [69] simulates the database of a wholesale company, which includes a number of warehouses. TPC-C consists of five types of transactions: the *New-Order* transaction simulates the procedure of entering an

Name	Wait	#Txn-Types	#WH	#Workers	Cross-WH (%)
Vanilla [69]	Yes	5	Configurable	10 per WH	9.5% and 15%
Calvin [60, 67, 68]	No	2	20 per server	4 per server	0-100%
Silo [63, 71]	No	5	1-32	1 per WH	0-100%
DrTM [26, 76]	No	5	8 per Server	1 per WH	0-100%
Janus [39, 53]	No	5	6	1-10K	9.5% and 15%
Cicada [17, 43]	No	5	1-28	1-28	9.5% and 15%
GAM [15, 30]	No	2	4 per server	1 per WH	0-100%
Star [46, 65]	No	2	12 per server	1 per WH	0-100%
Aria [10, 45]	No	2	1-180	1 per WH	0-100%

Table 1: TPC-C settings used by different systems.

Name	#KV pairs	KV size	R/W ratio	Distribution
Vanilla [78]	120M	1KB	50:50 or 95:5 or 100:0	Uniform/Zipfian (0.99)
Star [46, 65]	2.4M	8B/100B	90:10	Uniform
Silo [63, 71]	160M	8B/100B	80:20	Uniform
Cicada [17, 43]	10M	8B/100B	95:5/50:50	Zipfian
HERD [36, 40]	480M	16B/(4B - 1KB)	50:50	Uniform/Zipfian
TAPIR [81, 82]	1M	8B/1KB	50:50	Zipfian

Table 2: YCSB settings used by different systems.

order. Since *New-Order* transactions targeting the same warehouse need to read and update the same *next-order-ID* value, they are the major contention point of TPC-C. The *Payment* transaction updates the customer's balance and reflects the payment on the district and warehouse sales statistics. The other three types are less frequent so we skip their details.

Table 1 shows the common parameters tuned by different works: "Wait" identifies whether a customer will wait a while before starting the next transaction; "Txn-Types" identifiers how many types of transactions to run (5 for all types and 2 for *New-Order* and *Payment*); "#WH" identifies how many warehouses to use (also called "scale factor" in TPC-C); "#Workers" identifies how many concurrent customers may access a warehouse; and finally both *New-Order* and *Payment* have a chance to access a remote warehouse, and such chance is identified by "Cross-WH"[1].

Vanilla TPC-C only allows to tune the number of warehouses: with a long wait time per customer and a restriction of 10 customers per warehouse, vanilla TPC-C puts a very low limit on the throughput one can get per warehouse. Therefore, to get a high throughput, one needs to use many warehouses, which results in a low-contention and I/O heavy workload. Since many research prototypes focus on addressing contentions, they remove wait time and use a small number of warehouses to reach a high contention level. It is out of the scope of this paper to determine whether such tuning represents any realistic workload, since we cannot find any studies about transaction workload and it is unclear whether the 30-year old TPC-C can still represent a realistic workload today. As a result, an extensive evaluation is perhaps a safe choice.

For an extensive evaluation, we 1) do not use waiting since no systems use it; 2) try both 2 and 5 transaction types if the target system allows; 3) tune the number of warehouses from 1 to a large value allowed by DRAM (all target systems assume all data can fit into DRAM); 4) tune the number of workers from 1 to a large value; 5) tune cross-warehouse rate from 0% to 100%. We list detailed parameter values in Table 3, but note that some systems disallow the tuning of certain parameters.

Parameters of YCSB. Yahoo! Cloud Serving Benchmark (YCSB) is a benchmark primarily targeting NoSQL database systems with a key-value like interface [20]. The commonly tuned parameters include number of key-value pairs, size of a key-value pair, read/write ratio, and the distribution of keys.

Table 2 lists the parameter values used by the vanilla YCSB paper and those used by research prototypes. Furthermore, we find a number of studies about production key-value stores [11, 14, 55, 79], and these studies show that industrial companies can have a variety of workloads. To quote a few conclusions from the recent Twitter study [79]:

—"in addition to the well-known use case of serving read-heavy workloads, a substantial number of Twemcache clusters are used to serve write-heavy workloads."

—"a recent work from Facebook [14] suggested that in-memory caching workloads may not follow Zipfian distri-

[1]TPC-C uses different chances for *New-Order* and *Payment*, but since many prototypes tune them together, we only list one here.

bution."; "Measuring all Twemcache workloads, we observe the majority of the cache workloads still follow Zipfian distribution."; "most of the α values (of Zipfian distribution) are in the range from 1 to 2.5."

—"around 85% of Twemcache clusters have a mean key size smaller than 50 bytes, with a median smaller than 38 bytes"; "the mean value size falls in the range from 10 bytes to 10 KB, and 25% of workloads show value size smaller than 100 bytes, and median is around 230 bytes."

The versatility of production workloads, both within a single organization and across different organizations, is exactly one of the motivations for our pursuit of extensive evaluation: even if a work is optimized for one important workload, it would be helpful to know how it performs on other workloads.

For an extensive evaluation, we try to 1) tune the key-value size from 8B to 1KB; 2) tune the number of key-value pairs from 128K to the maximal allowed by DRAM; 3) tune read/write ratio from 100:0 to 0:100; and 4) try both uniform and Zipfian distribution and try different α values for the Zipfian distribution.

System parameters. For distributed systems, we measure system scalability by tuning the number of servers. For systems supporting replication, we also tune the number of replicas. However, due to resource limitations, we do not try a large number of servers.

Testbed. We run our experiments on CloudLab [18] and a local cluster. We try to use hardware that is close to the one used by the original paper. On CloudLab, we use m510 machines to run Aria experiments, c6320 machines to run Silo and Cicada experiments, rs630 to run Janus experiments, c220g1 machines to run TAPIR experiments, d430 to run Calvin experiments, c6220 machines to run Star and GAM experiments, and r320 machines to run HERD experiments. We test DrTM on a local cluster, in which each node is equipped with a 28-core Intel Xeon E5-2680 CPU and 128 GB of RAM. The cluster is interconnected by a 100 Gbps Mellanox CX-4 adapter. We use OpenFabrics Enterprise Distribution (OFED) version 4.9.

3 Sampling and Regression to Reduce Experiment Time

Cost of extensive evaluation. Given all the settings to measure and the length of each experiment, one can estimate the total machine hours necessary to run all experiments once. In our experience, two factors are adding uncertainties to such estimation: first, both TPC-C and YCSB have an initialization phase to populate data, which could take long, before running any experiments. Second, it is generally a recommended practice to run experiments multiple times to check whether the result is stable. In our experience, we find most results are stable so we decide to run three times, but we do observe a few systems have unstable performance under a subset of settings. In general, we observe unstable performance is often an indicator of underlying issues, so we should investigate the reason instead of simply running experiments more times.

Table 3 summarizes the settings we finish to run and the total machine hours we use to run each experiment once. In general, we observe that, perhaps not surprisingly, distributed systems require much more machine hours since they not only add a parameter (i.e., number of servers), but also require more machines to run these distributed experiments. As a result, for single-node systems, we find the total machine hours is well affordable, but for distributed systems, we have to carefully pick parameter values to minimize total machine hours. To be concrete, 1800 machine hours means 75 machines for a day; considering we need to run experiments three times and may need to run multiple iterations due to optimization or bug fix, we may need 75 machines for a week or longer, which is still manageable but could be a burden. If one needs to tune more parameters or more values of a parameter, the total machine hours required can quickly get out of control.

Sampling and regression. We study whether we can reduce the total machine hours with the classic sampling and regression idea: if we could run a subset of experiments and use the results to accurately predict the performance under the remaining settings, then we don't need to run all experiments.

We use the following methodology: for each system, from the results we have collected, we choose a subset of them as the training set and use the remaining results as the testing set; we train a model on the training set and test its accuracy on the testing set; we further tune the size of the training set to see how it affects the accuracy of prediction. We used scikit-learn [56] for our data analysis.

In general, regression does not work well if the values fall into a wide range, since most regression algorithms target minimizing the root-mean-square error (RMSE) between predicted and real values, and RMSE is biased toward high values (e.g., 10% error at a high value affects RMSE more than 10% error at a low value). To address this problem, instead of trying to predict the raw throughput, we try to predict the throughput per server core, which has a smaller value range.

We first try linear regression, and find it does not work very well in general, with a coefficient of determination (R^2) value lower than 0.6 in most cases. This is not surprising since many systems have non-linear throughput with regard to certain parameters (Section 5).

We then try a classic neural network based multilayer perceptron model (MLP) [31]. In our experiments, we use two hidden layers with 10 and 5 neural nodes, respectively. The weights between each layer are updated during the backpropagation by an optimization algorithm. We use the Limited-memory Broyden–Fletcher–Goldfarb–Shanno (L-BFGS) algorithm [44] as our optimizer, which performs better for small datasets. Our models are trained with 10,000 iterations. We

System	Settings	Machine hours
Calvin-TPCC	#WH={1,2,4...128},#Node={1,2,4...32},Cross-WH(%)={0,10...100}	832
Silo-TPCC	#WH={$1,2,4\ldots512$}, #Workers={1,2,4,8,16,28}, #Txn-Types={5,2,NewOrder-only},Cross-WH-NewOrder(%)={0,10 ... 100}	130
DrTM-TPCC	#Node={2,4,8}, #Workers={2,4,8,16,32}, #WHperNode={$1,2,4\ldots256$}, Cross-WH(%)={0,20 ... 100}	192
Janus-TPCC	#Clients={1,10,20,40,80,100,200,500,1000,2000,4000} #Replica={1,3,5,7}, #CPUperNode={1,2,4,8,16}, #WH={$1,2,4\cdots \mid \#WH \times \#Replica/\#CPUperNode \leq 25$}	1886
Cicada-TPCC	#WH={1,2,4...256},#Workers={1,2,4,8,16,28},#Txn-Types={5,2}	9
GAM-TPCC	#WH={2,4,8,16,32 / Node}, #Node={2,4,8}, Cross-WH(%)={0,20,40...100}, #Workers={1,2,4,8,16},#Txn-Types={2}	721
Star-TPCC	#WH={16,32,128,256},#Workers={1,2,4,8,16}, Cross-WH(%)={0,20,40...100},#Node={2,4,8}, #Txn-Types={2}	210
Aria-TPCC	#WH={1,2,4...1024},Cross-WH(%)={0,10...100},#Nodes={1,2,4,8} #Workers={1,2,4,8,12},#Locks={1,2,4,6|#Locks<#Workers}	360
Star-YCSB	#KV Pairs={3.2M, 6.4M, 25.6M, 51.2M }, #Workers={2,8,16}, Cross-partition(%)={0,20,40...100}, Zipfθ={0,0.4,0.8,0.99}, R/W Ratio={0/100,20/80,40/60,100/0}, #Node={2,4,8}	384
Silo-YCSB	#KV Pairs={128K,512K,2M, 8M, 32M,128M} #Workers={1,2,4,8,16,28},R/W Ratio={0/100,20/80,40/60,100/0}	5
Cicada-YCSB	#KV Pairs={128K,512K,2M, 8M, 32M},KV size={8B,32B,128B, 1KB}, #Workers={1,4,16,28}, Zipfθ={0,0.4,0.8,0.99}, R/W Ratio={0/100,20/80,40/60,100/0},batch size={1,10,100}	300

HERD-YCSB	#KV Pairs={6M, 24.5M, 96M, 384M, 1536M}, KV size={8B, 32B, 128B, 512B, 1KB},
	Zipfθ={0,0.4,0.8,0.99}, R/W Ratio={0/100,20/80,40/60,100/0}
		325

TAPIR-YCSB	#Clients={5,10,15,20,25},#KV Pairs={128K,512K,2M, 8M, 32M}, R/W Ratio={0/100,20/80,40/60,100/0}, (#shard,#replica)={(1,3),(1,5),(1,7),(2,3),(2,5),(3,3)}	1218

Table 3: The settings and machine hours we use for each system, including its competitors. Some systems don't support the tuning of a certain parameter. We set the length of each experiment to be the same as the one used by the original paper.

use this setting because it performs better than other settings we have tried, though note that this work is by no means an exhaustive search of the best model or setting.

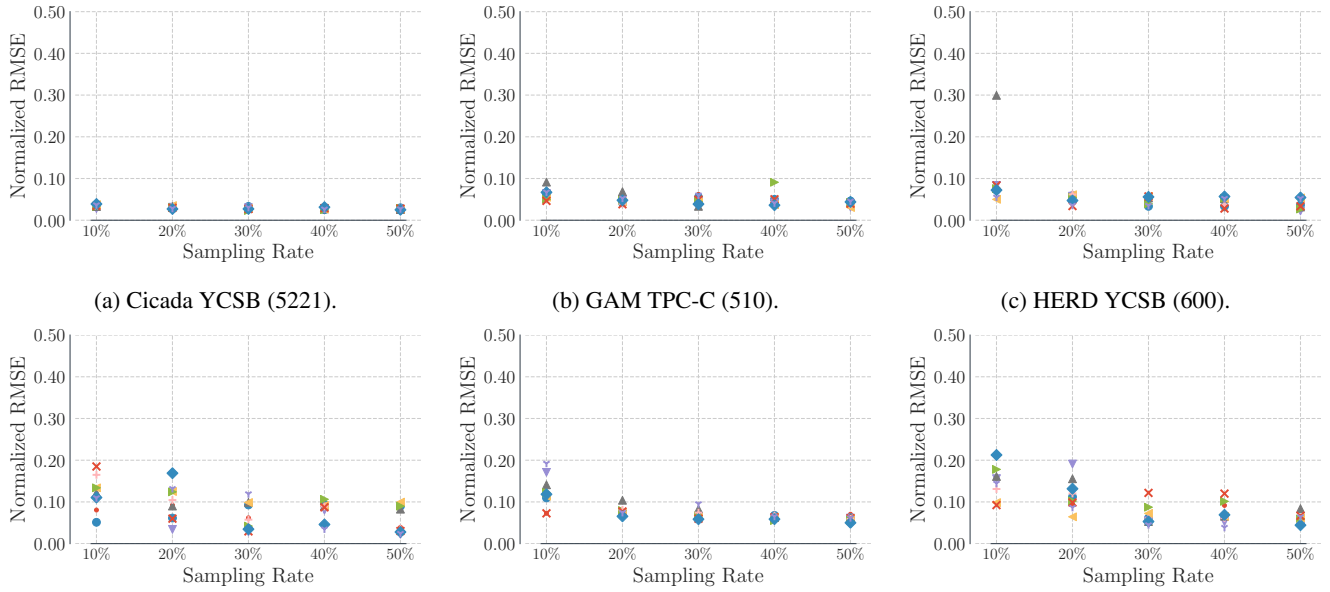
For non-linear models, we meet a challenge that there is no standard metric, like R^2 for linear models, to tell whether the prediction result is good or not: multiple articles argue R^2 is inappropriate for non-linear models [6, 41]; RMSE value depends on the value range and thus it is impossible to set a threshold. We finally decide to use $\frac{RMSE}{MAX_VALUE - MIN_VALUE}$ as a normalized RMSE value [3].

Figure 1 shows that most systems can achieve good accuracy (normalized RMSE smaller than or close to 0.1) given enough samples, which indicates sampling and regression is a promising idea to reduce experiment cost.

Figure 2 shows the three systems+benchmarks that have a higher normalized RMSE and we investigate the reasons. For Cicada TPC-C, the reason is that there are too few total data points (i.e., 106) as we can see the normalized RMSE of the prediction is still declining with more samples. In practice, this should not be a problem since we can simply run all experiments in this case. For Calvin, our investigation shows that it has performance anomalies at a number of settings due to implementation issues (Section 5). Aria is the most interesting case because, first, as shown in Figure 2c, some training sets can achieve very good accuracy but some do not, and second, our manual investigation finds its performance pattern is very regular. We tried different models and feature settings, and found that removing #threads from the feature list can significantly improve the prediction accuracy: in Figure 2c, with a 50% sampling rate, five training sets lead to normalized RMSE over 0.2 but after removing the #threads feature, only one training set leads to normalized RMSE over 0.2. Therefore we suspect its inaccuracy is caused by sub-optimal model and feature selection, which may cause problems like overfitting. This may be addressed by running multiple models together and using the best one.

Suggestions. In summary, sampling and regression could reduce total experiments to run if the system has predictable performance; it could help to identify performance anomalies for less predictable systems.

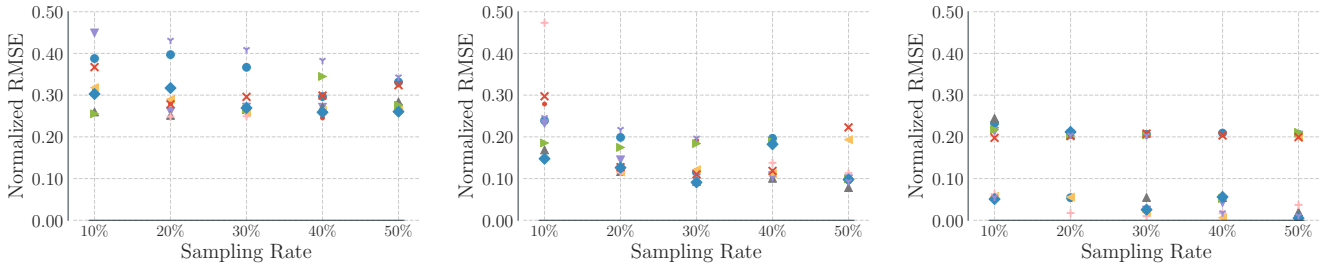
Based on our study, we make the following suggestion: we



(a) Cicada YCSB (5221). (b) GAM TPC-C (510). (c) HERD YCSB (600).

(d) DrTM TPC-C (342). (e) Janus TPC-C (1102). (f) Silo YCSB (144).

Figure 1: Examples of systems with accurate prediction. For each sampling rate, we try 10 times with different training sets, each corresponding to a point in these figures. The number in parenthesis is the total number of data points we have collected. Silo TPC-C, Star TPC-C, Star YCSB, and TAPIR YCSB figures have similar trends and can be found at our anonymous website [27].



(a) Cicada TPC-C (106). (b) Calvin (499). (c) Aria (605).

Figure 2: Systems with bad prediction accuracy. For each sampling rate, we try 10 times with different training sets, each corresponding to a point in these figures. The number in parenthesis is the total number of data points we collected.

can gradually test more settings and use maybe half of the results to predict the other half, until the prediction accuracy is good enough or becomes stable. In these cases, we can be confident that we have tested enough settings to capture the performance characteristics of the whole system. From our study, we suggest to use normalized RMSE as the metric and use 0.1 as a threshold, though we acknowledge both are debatable. If the prediction accuracy stays low at the end, it is usually an indicator of performance anomalies or inappropriate models, which requires further investigation.

Remaining Questions. This study does not answer what parameters to tune and what values to choose, which probably require a community-wide discussion. However, as a common software testing principle, testing the extreme values is usually a good idea to expose potential issues, as shown in Section 5.

This study also does not address the problem that we may not have enough machines. There are multiple works trying to evaluate a large cluster on a limited number of machines [8, 33, 62, 66, 75], which may be helpful to some extent.

4 Visualizing High-dimensional Data

When we measure a system with more than two parameters, the result becomes high-dimensional data that cannot be easily mapped to a 2D figure. One can select a subset of results to present but then we are going back to the question of which settings we should choose. Therefore, in this work, we set our goal to be presenting all data.

To achieve this goal, we design and develop a tool to visualize high-dimensional data on a figure called Tiled Heat

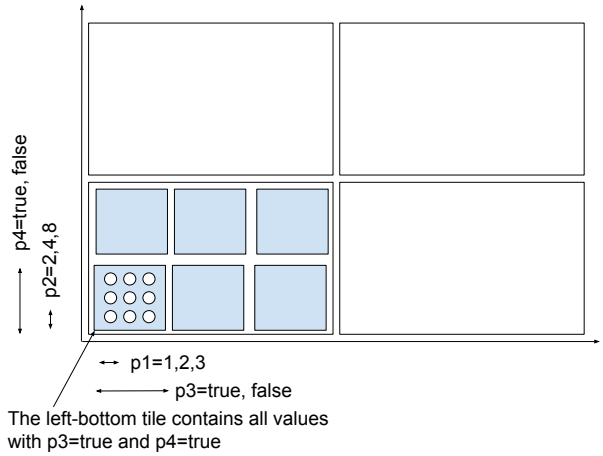


Figure 3: Tiled Heat Map.

Map (Figure 3): assuming the system has n parameters from p_1 to p_n, in the first iteration, our tool picks two parameters (e.g., p_1 and p_2) and groups all data with the same p_3 to p_n; then it is able to draw each group on a 2D figure called a "tile" since each group only contains two parameters p_1 and p_2. In the second iteration, our tool picks two additional parameters (e.g., p_3 and p_4) and groups tiles with the same p_5 to p_n into a super-tile. It repeats this procedure until it covers all parameters (the last iteration may just pick one parameter). Our tool uses Matplotlib [38] as its plotting interface. Section 5 shows some real Tiled Heat Maps we build.

Order of parameters. Our tool needs to choose parameters in each iteration. Among all the possible choices, we try to choose the one that causes the final figure to be "smooth" for easy reading (i.e., large values are close to each other and small values are close to each other). Formally, we define the following metric to measure how "smooth" a figure is, assuming there are n points in the figure, and each point has a coordinate (x_i, y_i) and a value v_i.

Definition 4.1. $smoothness = |SpearmanCorrelation(D,V)|$
, in which
$D = \{\forall(i,j), d_{ij} = |x_i - x_j| + |y_i - y_j|\}$
$V = \{\forall(i,j), v_{ij} = |v_i - v_j|\}$

We use Spearman's correlation coefficient [54], which is the Pearson's correlation [13] between two rank variables, to measure how well the relationship between D and V can be described using a monotonic function. We choose Spearman's correlation over Pearson's because the latter measures linearity, while we only care about monotonicity here.

Intuitively, a high smoothness value means the value difference between two points is highly correlated with the distance between them. In other words, points with very different values are far away from each other and points with close values are close to each other, which is exactly what we want.

Our tool tries all the possible orders of parameters to compute the smoothness of each order and chooses the one with the highest smoothness. If the figure contains many points, our tool samples a subset of them to compute the smoothness value. Assuming the target system has p parameters, there is a total of $p!$ choices: with a small p, iterating all choices is feasible: with a bigger p, we probably need to go for heuristic solutions. In our experiments, we find iterating all is feasible.

Choosing the value and color of each point. The Tiled Heat Map can be used to present different types of values, such as the raw throughput, the improvement of one system over another, etc.

In the Tiled Heat Map, we use a color to represent the value of each point. By default, our tool first finds the minimal and maximal values, and then evenly distributes all values among gray color #CECECE to #0A0A0A (we observe colors very close to full black or white are hard to distinguish, so we try to avoid them). However, just like most graph plotting tools, our tool allows the user to specify a color for a value range manually. In this paper, we use gray colors to meet the requirement of the conference, and using more colors can often create a more readable version.

Limitations. As shown in the next section, we find the Tiled Heat Map is good at presenting the overall trend about how parameter values (or a combination of parameter values) can affect system performance. However, it is not very good at presenting the absolute value of each data point or comparing values with a small difference (e.g., 10%). Therefore, we propose the Tiled Heat Map as a complement to traditional line or bar graphs. Furthermore, although the Tiled Heat Map can typically show more information on the same area, its capability is still limited by the number of pixels we can have: when viewing it on a computer, this problem can be addressed by zooming in/out, which we find helpful, but when printing it on paper, there is eventually going to be a limit. Finally, we have tried the Tiled Heat Map on data with at most five dimensions and it is likely that more dimensions will make the figure harder to read.

5 Case Studies

This section discusses our experience in evaluating prior prototypes under a wider range of settings, including both correctness and performance issues. We try our best to fix these issues, but due to time constraints and the nature of debugging, we are only able to fix a subset of them. Significant design-level changes are also out of the scope of this paper.

5.1 Correctness Issues

Configuration issues. Many systems have hard-coded configuration parameters, which prevent extensive evaluation.

- Fixed memory allocation. Silo, Cicada, DrTM, and GAM allocate a fixed amount of memory. As a result, they fail if the experiments load more data, even if there is enough physical memory. Fixing these issues is often not as simple as changing a value, since some systems need to allocate memory separately for different types of data (e.g., Silo allocates a memory region for its underlying B^+ tree nodes and database records, plus a buffer region for each thread), and we had to manually tune the size of each part to maximize the total amount of data they can support.

- Fixed thread/warehouse ratio. DrTM assumes the number of threads is equal to the number of warehouses. When we test with more warehouses, this leads to significant performance degradation. We fixed this issue by modifying DrTM source code to allow a thread to access more than one warehouse. Janus has the same constraint but due to time constraints, we have not fixed it.

- Too many open files. In Janus, we met a problem that the server processes fail to start when we start many servers. After debugging, we find the root cause is that, in Janus, every server thread needs to open a socket to every other server thread, so when starting many servers, the number of sockets on one server becomes larger than the default maximal number of open files allowed by Linux. While fixing this issue is simple (i.e., enlarge the maximal number of open files), such all-to-all socket creation may create scalability problems in a large deployment.

- Inappropriate timeout values. Many systems use timeout for different purposes, like detecting whether a node has failed or whether an experiment has failed. Under a new setting, however, the default timeout value sometimes becomes inappropriate, causing phenomena like a running experiment or a healthy node getting killed. We manually tune the timeout value when the default one does not work.

- Limit on message size. HERD relies on the RDMA inlining feature to improve performance, and this feature usually only allows messages smaller than 512B. This problem is admitted in the HERD paper, and a complete solution would be to design additional communication paths for larger messages. Since the additional path does not affect messages using the inlining feature, we have not implemented it.

Besides, a number of systems do not implement all the parameters we have tried. For example, TAPIR uses one thread per server, so we cannot tune the number of threads; many systems can only support two types of transactions in TPC-C. Fixing these issues requires much more effort and thus is out of the scope of this paper. Furthermore, some issues are caused by the inherent limitation of these works (e.g., in TPC-C, *New-Order* and *Payment* do not need range queries but the other three types need).

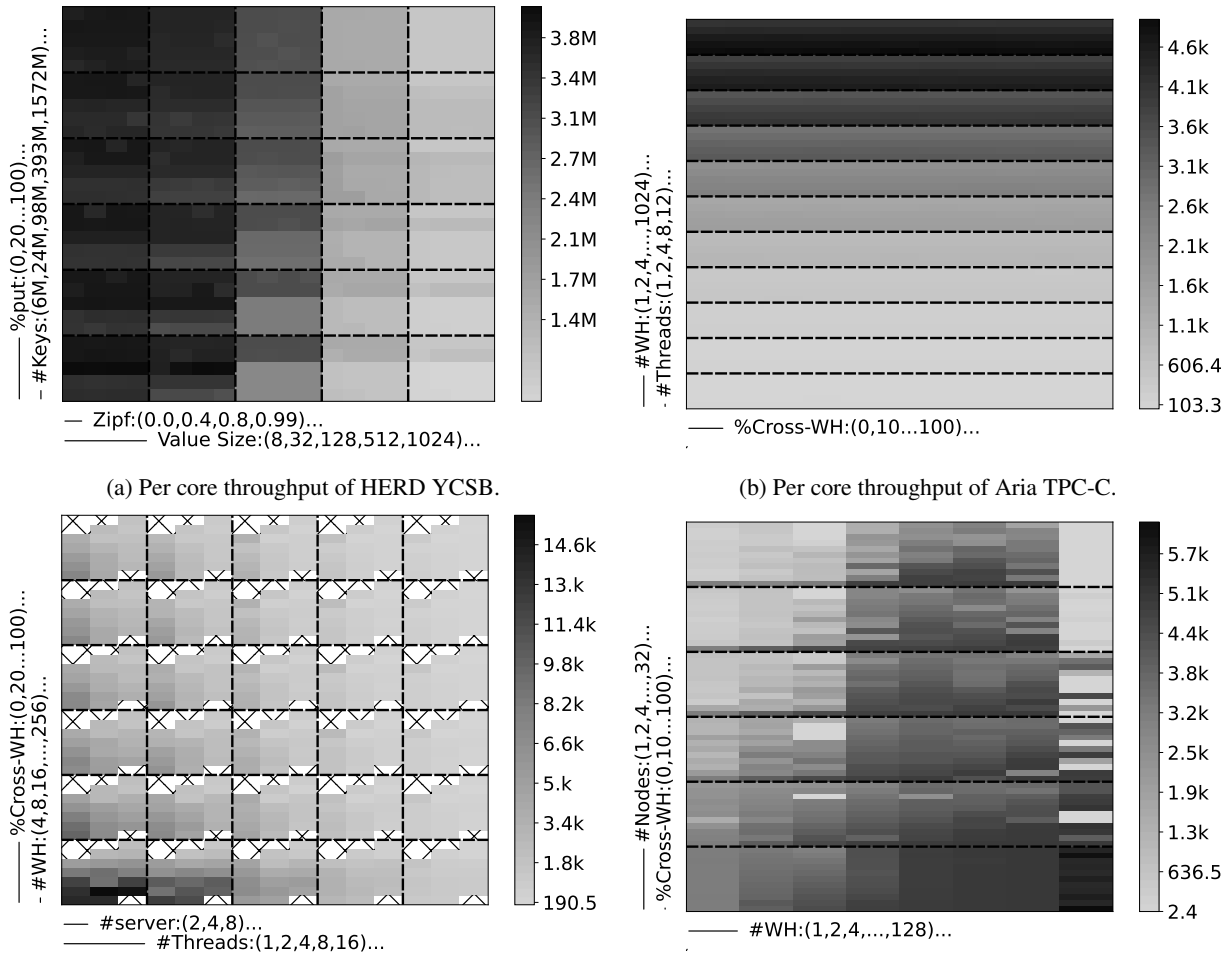
We make the following observations and suggestions: 1) it is often not hard to find a workaround after we know the root cause, but it sometimes takes longer to diagnose the root cause since the error messages often do not directly point to the root cause. Performing an extensive evaluation would help the researcher to identify and fix these issues, and help later works to compare to this work. 2) A fundamental solution to these problems, however, probably requires more thoughts (e.g., adaptive memory allocation, adaptive timeout, etc) and could be a research problem.

Buggy or Incomplete Implementation. A few other issues are caused by bugs or incomplete implementation. Due to the well-known difficulty of debugging non-deterministic issues, we have not been able to diagnose the root cause of all of them, and we report what we find.

- Segmentation fault. In Calvin, we find a number of settings cause abnormally low throughput, which further causes the bad prediction accuracy as shown in Section 3. Our investigation shows these settings cause a segmentation fault in the middle, but the experiments still calculate the throughput by dividing the number of finished transactions by the total experiment time, which results in very low throughput. We further investigate the reason for the segmentation fault and find it is caused by the inappropriate handling of NULL value returned by set lookup, which is more likely to happen under certain settings. We are able to fix this issue to avoid segmentation fault but we find our fix leads to other problems. Since we cannot find an easy fix and Aria has already implemented a more predictable version of Calvin, we finally give up fixing the original version of Calvin.

- Lock implementation. Silo implements a competitor called PartitionedStore as a comparison. When testing PartitionedStore with more settings, we find the experiments cannot finish under certain settings. Our investigation finally traces the problem to the customized lock implementation of PartitionedStore: sometimes a thread cannot acquire a lock even if the previous owner has released the lock. We are not able to fully locate the root cause but we find this problem is highly likely to be correlated with the data alignment in PartitionedStore: this problem only happens under settings whose total size of locks is a multiple of page size, and by artificially adding a few bytes, we can see the problem does not happen anymore.

- Incomplete recovery. In TAPIR experiments we find sometimes the experiments cannot finish. By looking at their logs, we find these experiments trigger a timeout in the middle, probably due to short timeout intervals, and then trigger failure recovery. But since TAPIR does not implement the recovery protocol, the whole system cannot make progress. As discussed above, we enlarge the timeout value to avoid triggering failure recovery.



(a) Per core throughput of HERD YCSB.

(b) Per core throughput of Aria TPC-C.

(c) Per core throughput of GAM TPC-C.

(d) Per core throughput of Calvin TPC-C.

Figure 4: Systems with stable and unstable performance (white X indicates settings we canot run).

This type of issue is perhaps unavoidable since it is unrealistic to expect a research prototype to be as stable and complete as a mature software and even a mature software can hardly be bug-free. We believe the bottom line is that, by extensive evaluation, we can be aware of these issues and have sufficient mechanisms to detect their manifestation. Of course, if a bug frequently happens, it is better to be fixed.

5.2 Comprehensive Performance Analysis and Comparison

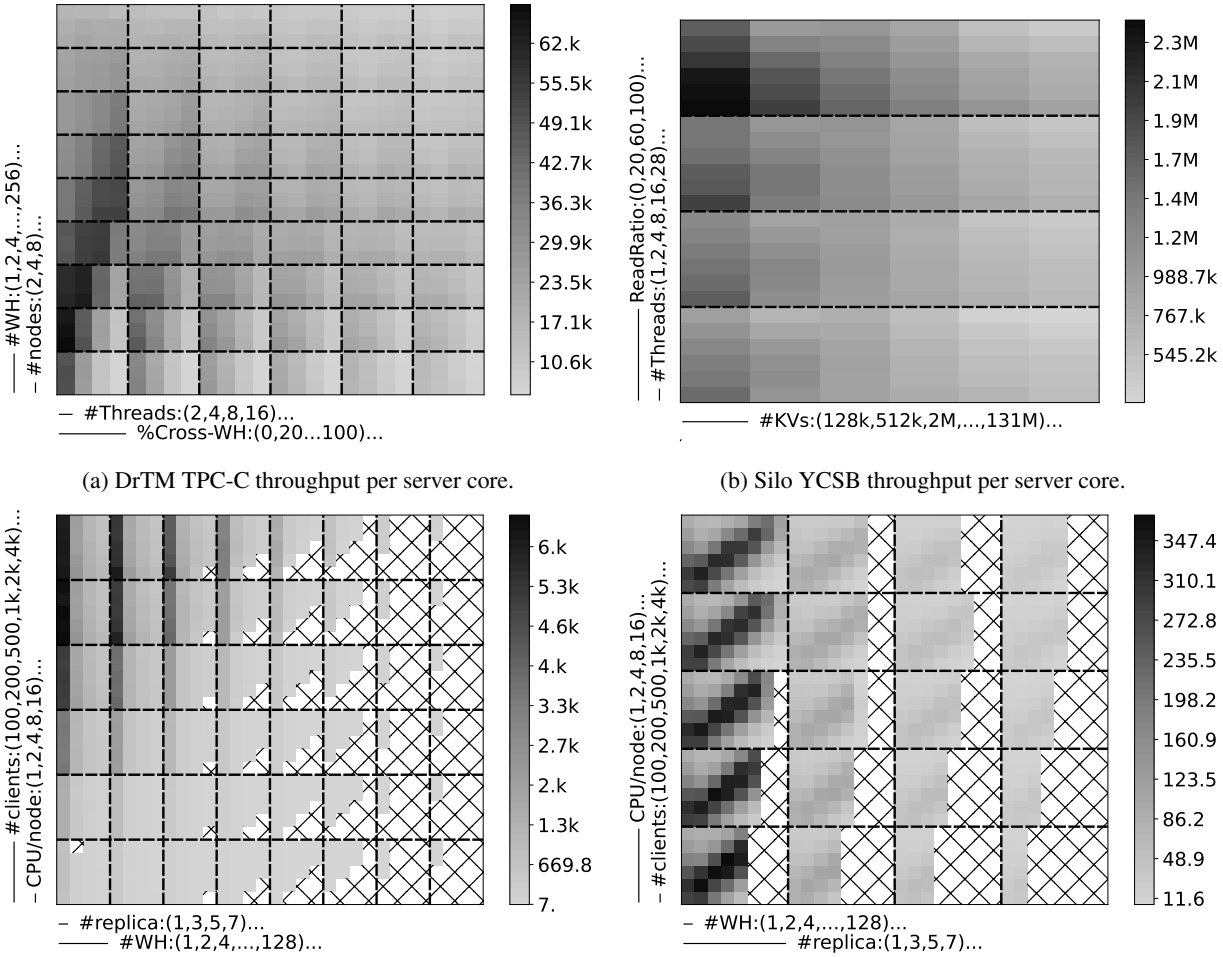
In this section, we present how extensive evaluation helps to reveal performance issues and allows a more comprehensive performance comparison. Due to space limitation, we discuss a subset of results here and publish all figures at our anonymous website [27].

Predictable vs unpredictable performance. As shown in Section 3, some systems have highly predictable performance and some are less predictable. The Tiled Heat Map can provide a direct illustration of such problems.

As shown in Figure 4, HERD has a very regular performance pattern: throughput decreases with larger KVs, which is as expected; more KVs improve the throughput by a small extent; other parameters do not seem to have a strong impact on throughput. Aria and GAM are two other examples with a very regular performance pattern: that's why we find it surprising that the prediction accuracy on Aria is not good (Section 3).

Calvin, on the other hand, has a less regular pattern as one can see there are often abrupt throughput changes between adjacent settings. By looking at its Tiled Heat Map, we investigate these abnormal points, which lead to our discovery in the previous section. The Calvin implementation from the Aria work has a much more regular pattern.

A Comprehensive Understanding of System Performance. Extensive evaluation with the Tiled Heat Graph has helped to reveal performance characteristics in a more



(a) DrTM TPC-C throughput per server core.

(b) Silo YCSB throughput per server core.

(c) Janus TPC-C throughput per server core.

(d) 2PL TPC-C throughput per server code.

Figure 5: Comprehensive analysis of system performance. White Xs indicate settings we cannot run due to resource constraints.

comprehensive manner and some of them are potential design issues.

Figure 5a shows the throughput of DrTM, which reveals several patterns: in general, more distributed transactions lead to lower throughput per core, which is expected; the number of nodes does not have a strong impact on throughput per core, which shows DrTM has good scalability by adding more nodes; the number of warehouses and the number of threads combined create the step-like pattern, because more threads than warehouses will create a high contention, which results in lower throughput. However, apart from all these explainable patterns, we find one pattern hard to explain: further increasing the number of warehouses beyond the number of threads causes a quite significant drop of throughput per core. We use *perf* [57] to profile DrTM and find, with more warehouses, DrTM spends more time on indexing. This is because DrTM uses tree indexes, and with more data, the trees grow taller and require more time for updating and searching.

Silo has exactly the same issue as shown in Figure 5b: it uses Masstree as the index and since the tree grows taller with more KVs, tree update takes longer, which causes performance degradation with more KVs.

A classic solution to this problem is to shard the data and use an index for each shard [16, 21, 42]. Actually among the systems we tested, HERD, Star, and even Silo's competitor PartitionedStore perform data partition as well. These examples show the risk without extensive evaluation: if a later work needs to compare with DrTM or Silo under a setting with more KVs, its author has to optimize the prior work for a fair comparison; even worse, if its author is unaware of these issues, s/he may make an unfair comparison. Later we will show how this issue affects the comparison between Silo and its competitor PartitionedStore.

Figure 5c and Figure 5d show the throughput of Janus and its competitor two-phase locking (2PL). They have the configuration constraint that $\#CPUsPerNode \times \#Nodes = \#replica \times \#WH$. In other words, increasing #WH will increase the number of nodes in these figures. As shown in Fig-

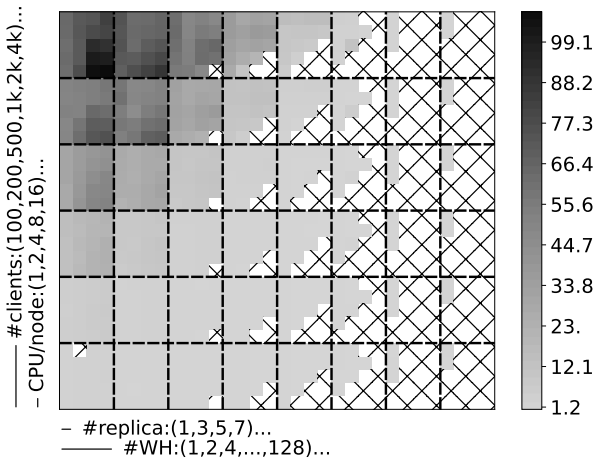


Figure 6: Janus over 2PL. White Xs indicate settings we cannot run due to resource constraint.

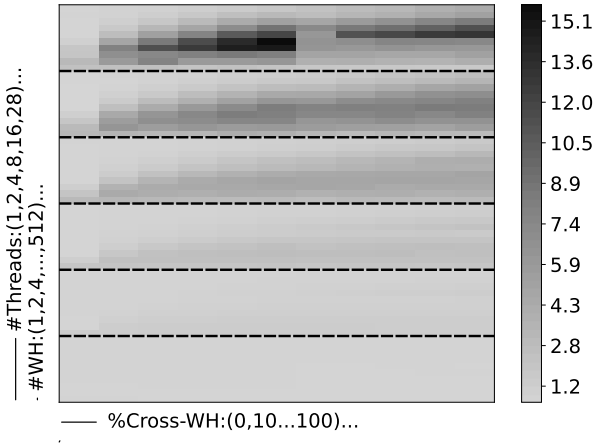


Figure 7: Silo over PartitionedStore

ure 5c, Janus' per-core throughput grows with more clients since too few clients will not be able to saturate the server; its per-core throughput drops with more replicas, which is expected since more replicas will incur more messages; its per-core throughput keeps stable when adding more CPUs per node, which means it scales up quite well; however, when adding nodes (i.e., warehouses), we can see its per-core throughput drops to some degree, which means it does not scale out very well. We investigate these experiments but do not find a clear bottleneck and we suspect the performance drop is caused by Janus' construction of a global dependency graph, which may incur more overhead when adding more nodes. How to improve the scalability of a global dependency graph could be an interesting research topic.

Figure 5d shows that 2PL has a very different pattern: its per-core throughput drops with more replicas, which is expected and the same as Janus; its combination of number of clients and number of warehouses shows the step pattern, which indicates 2PL can achieve the highest throughput if its #clients and #WH has a fixed rate. This is due to the fact that too few clients cannot saturate the server and too many clients will cause many deadlocks. Finally, we observe 2PL can both scale up and scale out pretty well, which is reasonable since 2PL has no inherent bottleneck.

A Comprehensive Comparison. Finally, we use two examples to show how extensive evaluation can allow a more comprehensive evaluation. To compare two works A and B, we use the Tiled Heat Map to draw throughput of A over throughput of B.

Figure 6 shows the comparison between Janus and 2PL. As one can see, Janus can achieve the highest improvement on the top left corner (i.e, more clients and fewer warehouses), which incurs the highest contention level; its improvement decreases with more warehouses, because as discussed above, Janus does not scale out very well but 2PL does. It would be interesting to see the trend in a large setting but unfortunately we do not have enough machines for that.

Figure 7 shows the comparison between Silo and its competitor PartitionedStore. In general, we can see the improvement drops with more warehouses, fewer threads, and fewer cross-warehouse transactions. Our analysis shows two reasons. First, Silo targets addressing contentions and contention level decreases with more warehouses, fewer threads, and fewer cross-warehouse transactions, so Silo's relative benefit becomes smaller. Second, as mentioned above, Silo does not partition data so its tree index grows taller with more data; PartitionedStore partitions data and thus does not have this problem. Among these two reasons, the first reason is the one we are interested in since Silo targets addressing contentions; the second one does not seem to be a fundamental limitation of Silo and is diminishing the true improvement of Silo.

Memory consumption. From the extensive evaluation, we observe the memory consumption of these systems vary a lot. We measure the memory consumption per warehouse under the TPC-C workload: for systems that perform customized memory allocation, we set a maximal memory limit and then increase the number of warehouses, till the system crashes; for systems relying on malloc, we use the *sysinfo* call to get memory consumption before and after loading a warehouse. Our measurement shows that their per-warehouse memory consumption varies significantly: 33MB for Calvin, 59MB for Aria, 64MB for Star and DrTM, 330MB for Silo, 360MB for Cicada, and over 600MB for GAM. Such gap is caused by the difference among their exact data to populate, their data layouts, and their index formats.

For in-memory systems, memory consumption is certainly an important factor and we suggest to at least report the memory consumption numbers. Of course how to reduce memory consumption can be a research topic as well.

Effects of Extensive Evaluation. Most of the issues discussed in this section are not obvious in the original settings

of their corresponding papers: DrTM assumes the number of threads is equal to the number of warehouses and tests with at most 28 warehouses, Silo uses a fixed 160M key-value pairs in YCSB and uses at most 32 warehouses in TPC-C. Therefore, for these two systems, the trend that more data lead to performance degradation is unclear. Janus tests with 6 warehouses, so the scalability issue is unclear as well. Extensive evaluation and proper visualization can help to reveal these problems and facilitate future comparison.

6 Summary of Suggestions

This section summarizes the suggestions:

- As shown in this work, extensive evaluation has the benefits of exposing both implementation and design issues. Since these benefits well align with the goals of artifact evaluation, maybe we should encourage extensive evaluation during artifact evaluation.
- Sampling and regression is a feasible solution to reduce the cost of extensive evaluation. We could gradually sample and test more settings until the prediction error becomes stable. If the error is still high, it may indicate performance anomalies, which require further investigation.
- To facilitate extensive evaluation, our community may have a discussion about tunable parameters and their values for popular benchmarks.

7 Related Work

Benchmarks. Many areas have well-established benchmarks for performance evaluation and comparison, such as the TPC series [70], SmallBank [9], YCSB [20], fio [28], the SPEC series [64], HPL [58], HPCG [25], Graph500 [32], MLPerf [50, 59], DAWNBench [19] and many others. Most of these benchmarks have restrictions about what parameters can be tuned and what values can be used for these parameters. While this work focuses on TPC-C and YCSB, we plan to perform a similar study on other benchmarks in the future.

Reproducibility. To improve code quality and facilitate future research, our community has encouraged research papers to verify their reproducibility through artifact evaluation [1, 2, 4, 5, 7]. In the meantime, many works study how to improve the reproducibility of research prototypes. For example, CloudLab studies the performance variability of its experiments and proposes guidelines to reduce such variability [49]; a recent work studies how cloud-based big data workload can be affected by network variability [73]; Fursin discussed the challenges and solutions from his experience of reproducing 150+ research papers [29]; Ursprung [61] studies how to improve the reproducibility of data science workloads. Compared to these efforts, our work further argues that, for the purpose of improving code quality and facilitating future research, we should encourage extensive evaluation to cover a wider range of settings.

Regression/Machine learning for systems. Regression or machine learning based methods have been gaining momentum in systems designs and optimizations. For instance, many studies [12, 72, 80] propose to use regression-based algorithms to tune systems configurations to achieve better performance. Recent systems research successfully exploits machine learning methods for resource management [23, 37, 47, 52], scheduling [22, 48], I/O optimizations [24, 34], and so on. Compared to these efforts, our work demonstrates that regression can be a feasible solution to reduce the cost of extensive evaluation.

Visualizing high-dimensional data. Visualizing high-dimensional data is a classic research topic with a large volume of work. However, most of them focus on how to find the structure within high-dimensional data and map it onto a 2D figure. This paper has a completely different goal and thus cannot re-use these works directly. We have tried two popular algorithms in this area t-SNE [74] and UMAP [51] on our data and do not find a meaningful output.

8 Conclusion

This paper shows that sampling and regression is a feasible solution to reduce the cost of extensive evaluation. Furthermore, it shows that extensive evaluation can help to reveal configuration issues, bugs, and design issues that do not manifest in the original settings and allow a comprehensive performance analysis. Based on such results, we make recommendations about how to improve artifact evaluation with extensive evaluation. We plan to explore other benchmarks in different areas, in particular those with more parameters, in the future.

Availability

We have published all figures on our anonymous website [27]. We will further publish our regression code, visualization tool, enhanced source code of prior works, and raw evaluation data.

References

[1] ACM SIGMOD Reproducibility. https://reproducibility.sigmod.org/.

[2] EuroSys Call for Artifacts. https://sysartifacts.github.io/eurosys2021/.

[3] How to normalize the RMSE. https://www.marinedatascience.co/blog/2019/01/07/normalizing-the-rmse/.

[4] OSDI Call for Artifacts. https://www.usenix.org/conference/osdi21/call-for-artifacts.

[5] PVLDB Reproducibility. http://vldb.org/pvldb/reproducibility/.

[6] R2 and nonlinear regression. https://www.graphpad.com/support/faq/r2-and-nonlinear-regression/.

[7] SOSP Call for Artifacts. https://sysartifacts.github.io/sosp2021/call.html.

[8] Nitin Agrawal, Leo Arulraj, Andrea C. Arpaci-Dusseau, and Remzi H. Arpaci-Dusseau. Emulating Goliath Storage Systems with David. In *FAST*, 2011.

[9] Mohammad Alomari, Michael Cahill, Alan Fekete, and Uwe Rohm. The cost of serializability on platforms that use snapshot isolation. In *2008 IEEE 24th International Conference on Data Engineering*, pages 576–585. IEEE, 2008.

[10] Aria source code. https://github.com/luyi0619/aria.

[11] Berk Atikoglu, Yuehai Xu, Eitan Frachtenberg, Song Jiang, and Mike Paleczny. Workload analysis of a large-scale key-value store. In *Proceedings of the 12th ACM SIGMETRICS/PERFORMANCE Joint International Conference on Measurement and Modeling of Computer Systems*, SIGMETRICS '12, pages 53–64, New York, NY, USA, 2012. ACM.

[12] Zhendong Bei, Zhibin Yu, Huiling Zhang, Wen Xiong, Chengzhong Xu, Lieven Eeckhout, and Shengzhong Feng. Rfhoc: A random-forest approach to auto-tuning hadoop's configuration. *IEEE Transactions on Parallel and Distributed Systems*, 27(5):1470–1483, 2016.

[13] Jacob Benesty, Jingdong Chen, Yiteng Huang, and Israel Cohen. Pearson correlation coefficient. In *Noise reduction in speech processing*, pages 1–4. Springer, 2009.

[14] Benjamin Berg, Daniel S. Berger, Sara McAllister, Isaac Grosof, Sathya Gunasekar, Jimmy Lu, Michael Uhlar, Jim Carrig, Nathan Beckmann, Mor Harchol-Balter, and Gregory R. Ganger. The CacheLib caching engine: Design and experiences at scale. In *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)*, pages 753–768. USENIX Association, November 2020.

[15] Qingchao Cai, Wentian Guo, Hao Zhang, Divyakant Agrawal, Gang Chen, Beng Chin Ooi, Kian-Lee Tan, Yong Meng Teo, and Sheng Wang. Efficient distributed memory management with rdma and caching. *Proc. VLDB Endow.*, 11(11):1604–1617, July 2018.

[16] Fay Chang, Jeffrey Dean, Sanjay Ghemawat, Wilson C. Hsieh, Deborah A. Wallach, Mike Burrows, Tushar Chandra, Andrew Fikes, and Robert E. Gruber. Bigtable: A Distributed Storage System for Structured Data. In *OSDI*, 2006.

[17] Cicada source code. https://github.com/efficient/cicada-exp-sigmod2017.

[18] Cloudlab. https://www.cloudlab.us/.

[19] Cody Coleman, Deepak Narayanan, Daniel Kang, Tian Zhao, Jian Zhang, Luigi Nardi, Peter Bailis, Kunle Olukotun, Chris Re, and Matei Zaharia. DAWNBench: An End-to-End Deep Learning Benchmark and Competition. https://cs.stanford.edu/ deepakn/assets/papers/dawnbench-sosp17.pdf, 2017.

[20] Brian F. Cooper, Adam Silberstein, Erwin Tam, Raghu Ramakrishnan, and Russell Sears. Benchmarking cloud serving systems with ycsb. In *Proceedings of the 1st ACM Symposium on Cloud Computing*, SoCC '10, pages 143–154, New York, NY, USA, 2010. ACM.

[21] Giuseppe DeCandia, Deniz Hastorun, Madan Jampani, Gunavardhan Kakulapati, Avinash Lakshman, Alex Pilchin, Swaminathan Sivasubramanian, Peter Vosshall, and Werner Vogels. Dynamo: Amazon's Highly Available Key-value Store. In *SOSP*, 2007.

[22] Christina Delimitrou and Christos Kozyrakis. Paragon: Qos-aware scheduling for heterogeneous datacenters. In *Proceedings of the Eighteenth International Conference on Architectural Support for Programming Languages and Operating Systems*, ASPLOS '13, page 77–88, New York, NY, USA, 2013. Association for Computing Machinery.

[23] Christina Delimitrou and Christos Kozyrakis. Quasar: Resource-efficient and qos-aware cluster management. In *Proceedings of the 19th International Conference on Architectural Support for Programming Languages and Operating Systems*, ASPLOS '14, page 127–144, New York, NY, USA, 2014. Association for Computing Machinery.

[24] Mo Dong, Tong Meng, Doron Zarchy, Engin Arslan, Yossi Gilad, Brighten Godfrey, and Michael Schapira. PCC vivace: Online-Learning congestion control. In *15th USENIX Symposium on Networked Systems Design and Implementation (NSDI 18)*, pages 343–356, Renton, WA, April 2018. USENIX Association.

[25] Jack Dongarra, Michael A Heroux, and Piotr Luszczek. High-performance conjugate-gradient benchmark: A new metric for ranking high-performance computing

systems. *The International Journal of High Performance Computing Applications*, 30(1):3–10, 2016.

[26] Drtm source code. https://github.com/SJTU-IPADS/drtm.

[27] Website for OSDI22 Paper#214. https://sites.google.com/view/osdi22-paper214.

[28] fio - flexible i/o tester. https://github.com/axboe/fio.

[29] Fursin, Grigori. Reproducing 150 Research Papers and Testing Them in the Real World: Challenges and Solutions. https://learning.acm.org/binaries/content/assets/leaning-center/webinar-slides/2021/grigorifursin_techtalk_slides.pdf, 2021.

[30] Gam source code. https://github.com/ooibc88/gam.

[31] Matt W Gardner and SR Dorling. Artificial neural networks (the multilayer perceptron)—a review of applications in the atmospheric sciences. *Atmospheric environment*, 32(14-15):2627–2636, 1998.

[32] Graph500 Committee. Graph500 benchmark. http://graph500.org.

[33] Diwaker Gupta, Kashi Venkatesh Vishwanath, and Amin Vahdat. DieCast: Testing Distributed Systems with an Accurate Scale Model. In *NSDI*, 2008.

[34] Mingzhe Hao, Levent Toksoz, Nanqinqin Li, Edward Edberg Halim, Henry Hoffmann, and Haryadi S. Gunawi. LinnOS: Predictability on unpredictable flash storage with a light neural network. In *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)*, pages 173–190. USENIX Association, November 2020.

[35] Rachael Harding, Dana Van Aken, Andrew Pavlo, and Michael Stonebraker. An evaluation of distributed concurrency control. *Proc. VLDB Endow.*, 10(5):553–564, January 2017.

[36] HERD source code. https://github.com/efficient/HERD.

[37] Henry Hoffmann. Jouleguard: Energy guarantees for approximate applications. In *Proceedings of the 25th Symposium on Operating Systems Principles*, SOSP '15, page 198–214, New York, NY, USA, 2015. Association for Computing Machinery.

[38] J. D. Hunter. Matplotlib: A 2d graphics environment. *Computing in Science & Engineering*, 9(3):90–95, 2007.

[39] Janus source code. https://github.com/NYU-NEWS/janus.

[40] Anuj Kalia, Michael Kaminsky, and David G. Andersen. Using rdma efficiently for key-value services. In *Proceedings of the 2014 ACM Conference on SIGCOMM*, SIGCOMM '14, pages 295–306, New York, NY, USA, 2014. Association for Computing Machinery.

[41] Tarald O. KvaLseth. Note on the R2 measure of goodness of fit for nonlinear models . *Bulletin of the Psychonomic Society volume*, 21:79–80, January 1983.

[42] Sangmin Lee, Zhenhua Guo, Omer Sunercan, Jun Ying, Thawan Kooburat, Suryadeep Biswal, Jun Chen, Kun Huang, Yatpang Cheung, Yiding Zhou, Kaushik Veeraraghavan, Biren Damani, Pol Mauri Ruiz, Vikas Mehta, and Chunqiang Tang. Shard manager: A generic shard management framework for geo-distributed applications. In *Proceedings of the ACM SIGOPS 28th Symposium on Operating Systems Principles*, SOSP '21, page 553–569, New York, NY, USA, 2021. Association for Computing Machinery.

[43] Hyeontaek Lim, Michael Kaminsky, and David G. Andersen. Cicada: Dependably fast multi-core in-memory transactions. In *Proceedings of the 2017 ACM International Conference on Management of Data*, SIGMOD '17, pages 21–35, New York, NY, USA, 2017. Association for Computing Machinery.

[44] Dong C Liu and Jorge Nocedal. On the limited memory bfgs method for large scale optimization. *Mathematical programming*, 45(1):503–528, 1989.

[45] Yi Lu, Xiangyao Yu, Lei Cao, and Samuel Madden. Aria: A fast and practical deterministic oltp database. *Proc. VLDB Endow.*, 13(12):2047–2060, July 2020.

[46] Yi Lu, Xiangyao Yu, and Samuel Madden. Star: Scaling transactions through asymmetric replication. *Proc. VLDB Endow.*, 12(11):1316–1329, July 2019.

[47] Martin Maas, David G. Andersen, Michael Isard, Mohammad Mahdi Javanmard, Kathryn S. McKinley, and Colin Raffel. Learning-based memory allocation for c++ server workloads. In *Proceedings of the Twenty-Fifth International Conference on Architectural Support for Programming Languages and Operating Systems*, ASPLOS '20, page 541–556, New York, NY, USA, 2020. Association for Computing Machinery.

[48] Hongzi Mao, Malte Schwarzkopf, Shaileshh Bojja Venkatakrishnan, Zili Meng, and Mohammad Alizadeh. Learning scheduling algorithms for data processing clusters. In *Proceedings of the ACM Special Interest Group on Data Communication*, SIGCOMM '19, page

270–288, New York, NY, USA, 2019. Association for Computing Machinery.

[49] Aleksander Maricq, Dmitry Duplyakin, Ivo Jimenez, Carlos Maltzahn, Ryan Stutsman, and Robert Ricci. Taming Performance Variability. In *13th USENIX Symposium on Operating Systems Design and Implementation (OSDI 18)*, pages 409–425, Carlsbad, CA, 2018. USENIX Association.

[50] Peter Mattson, Christine Cheng, Cody Coleman, Greg Diamos, Paulius Micikevicius, David Patterson, Hanlin Tang, Gu-Yeon Wei, Peter Bailis, Victor Bittorf, David Brooks, Dehao Chen, Debojyoti Dutta, Udit Gupta, Kim Hazelwood, Andrew Hock, Xinyuan Huang, Atsushi Ike, Bill Jia, Daniel Kang, David Kanter, Naveen Kumar, Jeffery Liao, Guokai Ma, Deepak Narayanan, Tayo Oguntebi, Gennady Pekhimenko, Lillian Pentecost, Vijay Janapa Reddi, Taylor Robie, Tom St. John, Tsuguchika Tabaru, Carole-Jean Wu, Lingjie Xu, Masafumi Yamazaki, Cliff Young, and Matei Zaharia. MLPerf Training Benchmark, 2020.

[51] Leland McInnes, John Healy, and James Melville. Umap: Uniform manifold approximation and projection for dimension reduction. *arXiv preprint arXiv:1802.03426*, 2018.

[52] Nikita Mishra, Connor Imes, John D. Lafferty, and Henry Hoffmann. *CALOREE: Learning Control for Predictable Latency and Low Energy*, page 184–198. Association for Computing Machinery, New York, NY, USA, 2018.

[53] Shuai Mu, Lamont Nelson, Wyatt Lloyd, and Jinyang Li. Consolidating Concurrency Control and Consensus for Commits under Conflicts. In *12th USENIX Symposium on Operating Systems Design and Implementation (OSDI 16)*, pages 517–532, Savannah, GA, 2016. USENIX Association.

[54] Leann Myers and Maria J Sirois. Spearman correlation coefficients, differences between. *Encyclopedia of statistical sciences*, 12, 2004.

[55] Rajesh Nishtala, Hans Fugal, Steven Grimm, Marc Kwiatkowski, Herman Lee, Harry C. Li, Ryan McElroy, Mike Paleczny, Daniel Peek, Paul Saab, David Stafford, Tony Tung, and Venkateshwaran Venkataramani. Scaling memcache at facebook. In *Proceedings of the 10th USENIX Symposium on Networked Systems Design and Implementation (NSDI 13)*, pages 385–398, Lombard, IL, 2013. USENIX.

[56] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and E. Duchesnay. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12:2825–2830, 2011.

[57] perf: Linux profiling with performance counters. https://perf.wiki.kernel.org.

[58] Petitet, A. and Whaley, R. C. and Dongarra, Jack and Cleary, A. Hpl - a portable implementation of the high-performance linpack benchmark for distributed-memory computers. https://www.netlib.org/benchmark/hpl/.

[59] Vijay Janapa Reddi, Christine Cheng, David Kanter, Peter Mattson, Guenther Schmuelling, Carole-Jean Wu, Brian Anderson, Maximilien Breughe, Mark Charlebois, William Chou, Ramesh Chukka, Cody Coleman, Sam Davis, Pan Deng, Greg Diamos, Jared Duke, Dave Fick, J. Scott Gardner, Itay Hubara, Sachin Idgunji, Thomas B. Jablin, Jeff Jiao, Tom St. John, Pankaj Kanwar, David Lee, Jeffery Liao, Anton Lokhmotov, Francisco Massa, Peng Meng, Paulius Micikevicius, Colin Osborne, Gennady Pekhimenko, Arun Tejusve Raghunath Rajan, Dilip Sequeira, Ashish Sirasao, Fei Sun, Hanlin Tang, Michael Thomson, Frank Wei, Ephrem Wu, Lingjie Xu, Koichi Yamada, Bing Yu, George Yuan, Aaron Zhong, Peizhao Zhang, and Yuchen Zhou. Mlperf inference benchmark. In *Proceedings of the ACM/IEEE 47th Annual International Symposium on Computer Architecture*, ISCA '20, page 446–459. IEEE Press, 2020.

[60] Kun Ren, Alexander Thomson, and Daniel J Abadi. An evaluation of the advantages and disadvantages of deterministic database systems. *Proceedings of the VLDB Endowment*, 7(10):821–832, 2014.

[61] Lukas Rupprecht, James C. Davis, Constantine Arnold, Yaniv Gur, and Deepavali Bhagwat. Improving reproducibility of data science pipelines through transparent provenance capture. *Proc. VLDB Endow.*, 13(12):3354–3368, August 2020.

[62] Rong Shi, Yifan Gan, and Yang Wang. Evaluating Scalability Bottlenecks by Workload Extrapolation. In *MASCOTS*, 2018.

[63] Silo source code. https://github.com/stephentu/silo.

[64] Standard Performance Evaluation Corporation. Standard performance evaluation corporation. https://www.spec.org/.

[65] Star source code. https://github.com/luyi0619/star.

[66] Cesar A. Stuardo, Tanakorn Leesatapornwongsa, Riza O. Suminto, Huan Ke, Jeffrey F. Lukman, Wei-Chiu Chuang, Shan Lu, and Haryadi S. Gunawi. ScaleCheck: A Single-Machine approach for discovering scalability bugs in large distributed systems. In *17th USENIX Conference on File and Storage Technologies (FAST 19)*, pages 359–373, Boston, MA, February 2019. USENIX Association.

[67] Alexander Thomson, Thaddeus Diamond, Shu-Chun Weng, Kun Ren, Philip Shao, and Daniel J. Abadi. Calvin code repository. https://github.com/yaledb/calvin.

[68] Alexander Thomson, Thaddeus Diamond, Shu-Chun Weng, Kun Ren, Philip Shao, and Daniel J. Abadi. Calvin: Fast Distributed Transactions for Partitioned Database Systems. In *Proceedings of the 2012 ACM SIGMOD International Conference on Management of Data*, SIGMOD '12, pages 1–12, New York, NY, USA, 2012. ACM.

[69] Transaction Processing Performance Council. The TPC-C home page. http://www.tpc.org/tpcc/.

[70] Transaction Processing Performance Council. Tpc-homepage. http://www.tpc.org.

[71] Stephen Tu, Wenting Zheng, Eddie Kohler, Barbara Liskov, and Samuel Madden. Speedy Transactions in Multicore In-memory Databases. In *Proceedings of the Twenty-Fourth ACM Symposium on Operating Systems Principles*, SOSP '13, pages 18–32, New York, NY, USA, 2013. ACM.

[72] Md. Wasi ur Rahman, Nusrat Sharmin Islam, Xiaoyi Lu, Dipti Shankar, and Dhabaleswar K. (DK) Panda. Mr-advisor: A comprehensive tuning, profiling, and prediction tool for mapreduce execution frameworks on hpc clusters. *Journal of Parallel and Distributed Computing*, 120:237–250, 2018.

[73] Alexandru Uta, Alexandru Custura, Dmitry Duplyakin, Ivo Jimenez, Jan Rellermeyer, Carlos Maltzahn, Robert Ricci, and Alexandru Iosup. Is big data performance reproducible in modern cloud networks? In *17th USENIX Symposium on Networked Systems Design and Implementation (NSDI 20)*, pages 513–527, Santa Clara, CA, February 2020. USENIX Association.

[74] Laurens Van der Maaten and Geoffrey Hinton. Visualizing data using t-sne. *Journal of machine learning research*, 9(11), 2008.

[75] Yang Wang, Manos Kapritsos, Lorenzo Alvisi, and Mike Dahlin. Exalt: Empowering Researchers to Evaluate Large-Scale Storage Systems. In *NSDI*, 2014.

[76] Xingda Wei, Jiaxin Shi, Yanzhe Chen, Rong Chen, and Haibo Chen. Fast in-memory transaction processing using rdma and htm. In *Proceedings of the 25th Symposium on Operating Systems Principles*, SOSP '15, pages 87–104, New York, NY, USA, 2015. ACM.

[77] Yingjun Wu, Joy Arulraj, Jiexi Lin, Ran Xian, and Andrew Pavlo. An empirical evaluation of in-memory multi-version concurrency control. *Proc. VLDB Endow.*, 10(7):781–792, March 2017.

[78] Yahoo! Research. Yahoo! Cloud Serving Benchmark. https://github.com/brianfrankcooper/YCSB.

[79] Juncheng Yang, Yao Yue, and K. V. Rashmi. A large scale analysis of hundreds of in-memory cache clusters at twitter. In *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)*, pages 191–208. USENIX Association, November 2020.

[80] Zhibin Yu, Zhendong Bei, and Xuehai Qian. *Datasize-Aware High Dimensional Configurations Auto-Tuning of In-Memory Cluster Computing*, page 564–577. Association for Computing Machinery, New York, NY, USA, 2018.

[81] Irene Zhang, Naveen Kr. Sharma, Adriana Szekeres, Arvind Krishnamurthy, and Dan R. K. Ports. Tapir code repository. https://github.com/UWSysLab/tapir.

[82] Irene Zhang, Naveen Kr. Sharma, Adriana Szekeres, Arvind Krishnamurthy, and Dan R. K. Ports. Building Consistent Transactions with Inconsistent Replication. In *SOSP*, 2015.